



Deep Reinforcement Learning

HOMEWORK 5 — MULTI-ARMED BANDITS

1 The ETC Algorithm with Heterogeneous Noise

Question 1: Pure Exploration Phase

[2 Points]

During the exploration phase, arm 1 is pulled m_1 times and arm 2 is pulled m_2 times. Determine the expected cumulative pseudo-regret over the exploration phase. Briefly explain why only the pulls of arm 2 contribute to the regret.

Answer:

By definition, the expected cumulative pseudo-regret during the pure exploration phase of total length $m = m_1 + m_2$ is formulated as the difference between the maximum possible expected reward and the expected reward obtained by the chosen policy. Given that arm 1 is uniquely optimal with $\mu^* = \mu_1$, the pseudo-regret equation is initialized as:

$$R_m = \mathbb{E} \left[\sum_{t=1}^{m_1+m_2} (\mu^* - \mu_{A_t}) \right] = \sum_{t=1}^{m_1+m_2} \mu_1 - \mathbb{E} \left[\sum_{t=1}^{m_1+m_2} X_t \right]$$

Furthermore, we can decompose the total reward summation by separating the steps where the optimal arm is pulled from the steps where the suboptimal arm is pulled. Expanding the expectation operator allows us to extract the underlying constant true means from the stochastic rewards:

$$\begin{aligned} R_m &= (m_1 + m_2)\mu_1 - \left(\mathbb{E} \left[\sum_{j=1}^{m_1} X_{1,j} \right] + \mathbb{E} \left[\sum_{k=1}^{m_2} X_{2,k} \right] \right) \\ &= (m_1 + m_2)\mu_1 - m_1\mu_1 - m_2\mu_2 \end{aligned}$$

Consequently, the terms associated with the optimal arm cancel out, isolating the mathematical expression to a stark subtraction of the suboptimal arm's total expected baseline from its corresponding optimal allotment:

$$R_m = m_2(\mu_1 - \mu_2) = m_2\Delta$$

Intuitively, only the pulls of arm 2 contribute to the cumulative pseudo-regret because choosing the optimal action (arm 1) yields an opportunity cost of zero relative to the theoretical oracle strategy. Moreover, each individual allocation to arm 2 incurs an expected performance penalty precisely equal to the suboptimality gap Δ , which demonstrates that exploration regret accumulates solely as a linear function of suboptimal selection rounds.

Question 2: Arm Selection Probability

[2 Points]

Prove the following upper bound on the probability of selecting the suboptimal arm:

$$\mathbb{P}(\hat{\mu}_2 \geq \hat{\mu}_1) \leq \exp \left(-\frac{\Delta^2}{2\left(\frac{\sigma_1^2}{m_1} + \frac{\sigma_2^2}{m_2}\right)} \right)$$

Briefly discuss how this probability changes as m_1 , m_2 , σ_1 , σ_2 , or Δ changes.

Answer:

To evaluate the probability of committing to the suboptimal arm at the conclusion of the exploration phase, we analyze the difference between the empirical means. Let $Z = \hat{\mu}_2 - \hat{\mu}_1$ define this combined random variable. Given that the individual rewards are sampled independently, the empirical mean $\hat{\mu}_1$ is $\sigma_1/\sqrt{m_1}$ -sub-Gaussian and $\hat{\mu}_2$ is $\sigma_2/\sqrt{m_2}$ -sub-Gaussian. Consequently, their linear combination inherits a sub-Gaussian profile with a cumulative variance proxy parameter given by:

$$\sigma_Z^2 = \frac{\sigma_1^2}{m_1} + \frac{\sigma_2^2}{m_2}$$

Furthermore, the mathematical expectation of this combined variable is centered around the negative suboptimality gap:

$$\mathbb{E}[Z] = \mathbb{E}[\hat{\mu}_2] - \mathbb{E}[\hat{\mu}_1] = \mu_2 - \mu_1 = -\Delta$$

The algorithm commits to the suboptimal arm precisely under the condition that $Z \geq 0$. By centering the variable around its expected value, this relational condition can be equivalently structured as a standard tail probability problem:

$$\mathbb{P}(Z \geq 0) = \mathbb{P}(Z - \mathbb{E}[Z] \geq \Delta)$$

Applying Hoeffding's inequality for general sub-Gaussian random variables directly yields the required analytical upper bound:

$$\mathbb{P}(\hat{\mu}_2 \geq \hat{\mu}_1) = \mathbb{P}(Z - \mathbb{E}[Z] \geq \Delta) \leq \exp \left(-\frac{\Delta^2}{2\sigma_Z^2} \right) = \exp \left(-\frac{\Delta^2}{2\left(\frac{\sigma_1^2}{m_1} + \frac{\sigma_2^2}{m_2}\right)} \right)$$

Intuitively, the analytical behavior of this probability bound under parameter variations is summarized below:

- ◇ **Exploration Allocations** (m_1, m_2): Increasing the number of exploration rounds reduces the aggregate variance proxy σ_Z^2 . Consequently, the denominator within the exponent shrinks, which increases the absolute magnitude of the negative exponent and drives the overall error probability down.
- ◇ **Noise Parameters** (σ_1, σ_2): Larger sub-Gaussian parameters signify heightened environmental noise. This increases σ_Z^2 , which inflates the denominator, dampens the negative exponent, and subsequently increases the probability of choosing the wrong arm.
- ◇ **Suboptimality Gap** (Δ): A larger value of Δ translates to a wider performance separation between the two arms. This increases the numerator Δ^2 , forcing the exponential upper bound to decay rapidly toward zero, thereby significantly minimizing the risk of misidentification.

Question 3: Combining The Results

[2 Points]

Combine the results from the last two questions to obtain an upper bound on the expected pseudo-regret R_n of the modified ETC algorithm. Your bound should be written in terms of n , m_1 , m_2 , Δ , σ_1 , and σ_2 . Discuss the trade-off between exploration and commitment. What happens if m_1 and m_2 are too small? What happens if they are too large?

Answer:

The total expected pseudo-regret R_n over a horizon of n rounds is decomposed into the sum of the expected regret incurred during the pure exploration phase and the expected regret accumulated throughout the subsequent commitment phase. Consequently, utilizing the linear exploration cost derived in Question 1 and the misidentification probability bounded in Question 2, we formalize the global pseudo-regret expression as follows:

$$\begin{aligned} R_n &= R_{\text{explore}} + R_{\text{commit}} \\ &= m_2 \Delta + (n - m_1 - m_2) \Delta \cdot \mathbb{P}(\hat{\mu}_2 \geq \hat{\mu}_1) \end{aligned}$$

Substituting the sub-Gaussian tail bound directly into this decomposition yields a comprehensive analytical upper bound for the modified ETC policy:

$$R_n \leq m_2 \Delta + (n - m_1 - m_2) \Delta \exp \left(- \frac{\Delta^2}{2 \left(\frac{\sigma_1^2}{m_1} + \frac{\sigma_2^2}{m_2} \right)} \right)$$

Furthermore, this explicit mathematical formulation encapsulates the fundamental exploration-exploitation trade-off, which can be evaluated through two distinct operational extremes:

- ◇ **Insufficient Exploration (m_1 and m_2 are too small):** When the exploration allocations are inadequate, the empirical variances $\frac{\sigma_i^2}{m_i}$ remain substantially high. Consequently, the exponential term fails to decay, leaving a severe risk of misidentifying the true optimal arm. Intuitively, the algorithm suffers a high probability of committing to the suboptimal arm for the vast majority of the remaining $(n - m_1 - m_2)$ rounds, forcing the cumulative regret to scale linearly with the horizon n .
- ◇ **Excessive Exploration (m_1 and m_2 are too large):** Conversely, if the exploration budgets are overly inflated, the probability of selecting the wrong arm approaches zero. However, the linear term $m_2 \Delta$ increases monotonically with the prolonged duration of the exploration phase. In the limit where $m_1 + m_2 \approx n$, the algorithm wastes critical rounds pulling a known inferior arm, causing the deterministic exploration regret to dominate the total loss profile.

Question 4: Optimal Exploration Allocation

[2 Points]

Find the optimal ratio $\frac{m_1}{m_2}$ that minimizes $\frac{\sigma_1^2}{m_1} + \frac{\sigma_2^2}{m_2}$. Then express the optimal values of m_1 and m_2 in terms of m , σ_1 , and σ_2 .

Answer:

To optimize the budget allocation under a fixed total exploration budget $m = m_1 + m_2$, we seek to minimize the variance proxy term that dictates the tail probability of a commitment error. We can establish a tight, fundamental lower bound for this objective function by multiplying

the target expression by the constant budget summation and applying the Cauchy-Schwarz inequality:

$$\left(\frac{\sigma_1^2}{m_1} + \frac{\sigma_2^2}{m_2}\right)(m_1 + m_2) \geq \left(\sqrt{\frac{\sigma_1^2}{m_1}} \cdot \sqrt{m_1} + \sqrt{\frac{\sigma_2^2}{m_2}} \cdot \sqrt{m_2}\right)^2$$

Consequently, simplifying the inner products on the right-hand side yields a constant lower bound that is independent of the individual selection variables m_1 and m_2 :

$$\left(\frac{\sigma_1^2}{m_1} + \frac{\sigma_2^2}{m_2}\right)(m) \geq (\sigma_1 + \sigma_2)^2 \implies \frac{\sigma_1^2}{m_1} + \frac{\sigma_2^2}{m_2} \geq \frac{(\sigma_1 + \sigma_2)^2}{m}$$

Furthermore, the minimum value of this variance expression is achieved precisely when equality holds in the Cauchy-Schwarz inequality. This algebraic equality condition necessitates a strict proportionality between the corresponding components, which dictates that:

$$\frac{\left(\frac{\sigma_1}{\sqrt{m_1}}\right)}{\sqrt{m_1}} = \frac{\left(\frac{\sigma_2}{\sqrt{m_2}}\right)}{\sqrt{m_2}} \implies \frac{\sigma_1}{m_1} = \frac{\sigma_2}{m_2} \implies \frac{m_1}{m_2} = \frac{\sigma_1}{\sigma_2}$$

Moreover, we can compute the explicit optimal allocations m_1^* and m_2^* by incorporating the global budget constraint $m_1 + m_2 = m$. Substituting the optimal ratio $m_1 = m_2 \frac{\sigma_1}{\sigma_2}$ into the constraint equation leads to the following formulation:

$$m_2^* \left(\frac{\sigma_1}{\sigma_2} + 1\right) = m \implies m_2^* = \frac{\sigma_2}{\sigma_1 + \sigma_2} m$$

Consequently, by mathematical symmetry, the optimal number of exploration rounds allocated to the first arm is uniquely determined as:

$$m_1^* = \frac{\sigma_1}{\sigma_1 + \sigma_2} m$$

Intuitively, this allocation strategy dictates that the arm possessing a higher level of sub-Gaussian noise must receive a proportionally larger share of the exploration budget, thereby ensuring that the empirical variance of both arms decays at a balanced rate.

Question 5: Optimal Number of Exploration Rounds

[2 Points]

Assume that Δ is known. Use the optimal allocation from Question 4 to choose the total exploration budget m that minimizes the pseudo-regret upper bound from Question 3. Show that, up to universal constants and lower-order terms, the optimized regret scales as $O\left(\frac{(\sigma_1 + \sigma_2)^2}{\Delta} \log n\right)$. Explain why this result is intuitive: why should the total amount of exploration increase when the noise levels σ_1 and σ_2 are larger, and decrease when the gap Δ is larger?

Answer:

To derive the optimal total exploration budget m^* , we substitute the minimized variance proxy from Question 4, which equals $\frac{(\sigma_1 + \sigma_2)^2}{m}$, along with the optimal allocation for the second arm $m_2^* = \frac{\sigma_2}{\sigma_1 + \sigma_2} m$, into the comprehensive regret bound established in Question 3. Consequently using Upper-Bounding Trick (get rid of $-m$), the global objective function to minimize with respect to m becomes:

$$R_n \leq \left(\frac{\sigma_2}{\sigma_1 + \sigma_2}\right) m \Delta + n \Delta \exp\left(-\frac{\Delta^2 m}{2(\sigma_1 + \sigma_2)^2}\right)$$

Furthermore, to find the critical value of m that minimizes this upper bound, we compute the first derivative with respect to m and set the resulting expression to zero:

$$\frac{\partial R_n}{\partial m} = \left(\frac{\sigma_2}{\sigma_1 + \sigma_2} \right) \Delta - n\Delta \left(\frac{\Delta^2}{2(\sigma_1 + \sigma_2)^2} \right) \exp \left(-\frac{\Delta^2 m}{2(\sigma_1 + \sigma_2)^2} \right) = 0$$

Isolating the exponential term yields the following definitive normalization relationship:

$$\exp \left(-\frac{\Delta^2 m}{2(\sigma_1 + \sigma_2)^2} \right) = \frac{2\sigma_2(\sigma_1 + \sigma_2)}{n\Delta^2}$$

Taking the natural logarithm on both sides allows us to solve explicitly for the optimal total exploration budget m^* :

$$m^* = \frac{2(\sigma_1 + \sigma_2)^2}{\Delta^2} \ln \left(\frac{n\Delta^2}{2\sigma_2(\sigma_1 + \sigma_2)} \right)$$

Consequently, expanding the logarithmic components isolates the dominant horizon parameter, showing that $m^* = \frac{2(\sigma_1 + \sigma_2)^2}{\Delta^2} \ln n - \mathcal{O} \left(\frac{(\sigma_1 + \sigma_2)^2}{\Delta^2} \ln \left(\frac{\sigma_2(\sigma_1 + \sigma_2)}{\Delta^2} \right) \right)$. Substituting this optimized budget m^* back into the objective function yields:

$$\begin{aligned} R_n &\leq \left(\frac{\sigma_2}{\sigma_1 + \sigma_2} \right) \left[\frac{2(\sigma_1 + \sigma_2)^2}{\Delta^2} \ln \left(\frac{n\Delta^2}{2\sigma_2(\sigma_1 + \sigma_2)} \right) \right] \Delta + n\Delta \left(\frac{2\sigma_2(\sigma_1 + \sigma_2)}{n\Delta^2} \right) \\ &= \frac{2\sigma_2(\sigma_1 + \sigma_2)}{\Delta} \ln n + \frac{2\sigma_2(\sigma_1 + \sigma_2)}{\Delta} \end{aligned}$$

Moreover, because the relationship $\sigma_2 \leq \sigma_1 + \sigma_2$ holds structurally, the leading coefficient directly matches the required scaling behavior. Consequently, up to universal constants and lower-order terms, the optimized cumulative pseudo-regret scales precisely as:

$$R_n = O \left(\frac{(\sigma_1 + \sigma_2)^2}{\Delta} \log n \right)$$

Intuitively, this mathematical scaling behavior reflects the fundamental information-theoretic limits of multi-armed bandits under asymmetric noise conditions:

- ◊ **Impact of Noise Levels (σ_1, σ_2) :** When the environmental sub-Gaussian noise parameters are large, the underlying true rewards are severely obscured by high stochastic variance. Consequently, the learner requires a significantly larger number of samples to construct a statistically confident estimate of the empirical means, which necessitates a larger exploration phase.
- ◊ **Impact of the Suboptimality Gap (Δ) :** Conversely, when the gap Δ is wide, the reward distributions of the two arms are highly distinct and separated. Intuitively, even noisy empirical averages can easily differentiate the superior arm from the inferior one, allowing the algorithm to reduce exploration rounds and commit early without incurring a high risk of error.

2 How to Model with Bandits?

Question 1: Adversarial Bandits

[2 Points]

Explain the fundamental difference between the definition of pseudo-regret in a stochastic bandit environment versus an adversarial bandit environment. Why is the stochastic definition inadequate in the presence of an adversary?

Answer:

The primary distinction between stochastic and adversarial bandit frameworks lies in the assumptions governing reward generation and the corresponding benchmark used to define regret. In a stochastic setting, rewards are drawn from static underlying probability distributions with fixed, time-invariant expectations $\mu_a = \mathbb{E}[X_{t,a}]$. Consequently, the performance baseline is anchored to the absolute best arm in expectation:

$$R_n = n \cdot \max_{a \in \mathcal{A}} \mu_a - \mathbb{E} \left[\sum_{t=1}^n \mu_{A_t} \right]$$

Conversely, the adversarial framework drops all distributional assumptions entirely, allowing the reward sequence $\{x_{t,a}\}_{t=1}^n$ to be completely arbitrary or chosen maliciously by an opponent. Because static expected means do not exist in this environment, the learner cannot benchmark performance against a theoretical distribution. Instead, the best achievable alternative is to evaluate the cumulative rewards retrospectively. The optimal baseline is thus defined as the maximum total reward that could have been achieved by committing to the single best static action in hindsight:

$$R_n(\pi, x) = \max_{a \in \mathcal{A}} \sum_{t=1}^n x_{t,a} - \mathbb{E} \left[\sum_{t=1}^n x_{t,A_t} \right]$$

Furthermore, the stochastic definition is entirely inadequate in the presence of an adversary because the concept of a stationary mean μ_a becomes mathematically ill-defined when rewards change arbitrarily at each step. Moreover, a strategic adversary can engineer non-stationary reward paths specifically designed to exploit algorithms that assume statistical consistency. Consequently, tracking performance against a fixed distribution fails, which necessitates a shift to the deterministic, hindsight-based benchmark of adversarial regret.

Question 2: The Adversary

[2 Points]

What role does the adversary play in the definition of adversarial regret? Consider a specific scenario where the adversary assigns identical rewards to all arms at every time step t (i.e., $x_{t,a} = c_t$ for all $a \in \mathcal{A}$). Mathematically derive the resulting regret and discuss the implications of this scenario on the learner's policy.

Answer:

The adversary plays the role of a malicious environment that determines the sequence of reward vectors in order to maximize the learner's cumulative pseudo-regret. By intentionally structuring these payoffs based on the algorithm's vulnerabilities, the adversary forces the learner to design robust, non-deterministic selection strategies. To analyze the specific edge case where the adversary assigns uniform rewards $x_{t,a} = c_t$ to all arms $a \in \mathcal{A}$ at each time step t , we substitute this condition directly into both terms of the adversarial regret formula. Consequently, the hindsight baseline for the single best static arm simplifies as follows:

$$\max_{a \in \mathcal{A}} \sum_{t=1}^n x_{t,a} = \max_{a \in \mathcal{A}} \sum_{t=1}^n c_t = \sum_{t=1}^n c_t$$

Furthermore, the expected reward accumulated by the learner's online policy π over the horizon n can be expanded linearly. Since every available action yields the exact same payoff c_t at step

t , the choice of the selected arm A_t becomes mathematically irrelevant to the summation:

$$\mathbb{E} \left[\sum_{t=1}^n x_{t,A_t} \right] = \mathbb{E} \left[\sum_{t=1}^n c_t \right] = \sum_{t=1}^n c_t$$

Consequently, subtracting the expected policy performance from the retrospective optimal baseline yields a total pseudo-regret that is identically zero:

$$R_n(\pi, x) = \sum_{t=1}^n c_t - \sum_{t=1}^n c_t = 0$$

Intuitively, this result demonstrates that if an adversary fails to create variation or asymmetric utility across the action space, the learner incurs no opportunity cost regardless of its underlying decision-making policy. Since no individual arm is suboptimal relative to another, even a completely arbitrary, static, or random policy achieves perfect performance against the hindsight benchmark. This highlights that adversarial regret fundamentally measures the learner's inability to exploit structural differences between actions; when the adversary eliminates these differences, the learning challenge collapses entirely.

Question 3: The Learner

[2 Points]

Can we deploy deterministic stochastic bandit algorithms, such as ETC or UCB, directly in an adversarial environment? Explain how a malicious adversary can exploit the deterministic nature of these algorithms to maximize the learner's regret.

Answer:

No, deterministic stochastic bandit algorithms like ETC or UCB cannot be directly deployed in adversarial environments. Because these algorithms lack randomization, their choices are fully predictable if the history of rewards is known. A strategic adversary can exploit this predictability to construct a worst-case reward sequence that forces the algorithm to perform poorly.

For instance, at any time step t , the adversary can compute exactly which arm A_t a deterministic policy will select based on past data. By setting a low reward for the chosen arm and a high reward for the unchosen arms, the adversary ensures that the learner suffers a constant regret at each step.

The specific failure mechanisms for these algorithms are:

- ◊ **ETC Failure:** During the exploration phase, the adversary can make a suboptimal arm appear highly rewarding. Once the fixed exploration period ends and ETC commits to this arm, the adversary can drop its reward to zero. Since ETC completely stops exploring after the commitment phase, it will continue to pull the low-reward arm for the remaining rounds, leading to a linear cumulative regret of $\mathcal{O}(n)$.
- ◊ **UCB Failure:** UCB relies on the assumption that rewards are independent and identically distributed (i.i.d.) over time to build its upper confidence bounds. In an adversarial setting, the environment is non-stationary and adaptive. If the adversary systematically feeds low rewards to the arm with the highest UCB score, the algorithm's confidence intervals become misleading. The past history no longer reflects future payoffs, breaking the mathematical foundation of the algorithm and preventing it from minimizing regret.

Therefore, randomization is an essential component in adversarial bandit algorithms (such as Exp3) to prevent the environment from predicting and exploiting the learner's decisions.

Question 4: The Objective

[2 Points]

What constitutes a successful strategy in an adversarial bandit environment? Provide the mathematical intuition behind achieving sublinear regret and present a concrete real-world application of this framework.

Answer:

A successful strategy in an adversarial bandit environment must rely on explicit randomization. Instead of choosing a single arm deterministically, the learner maintains a dynamically updated probability distribution over the available action space, as seen in algorithms like Exp3. Although the adversary can analyze the algorithm and discover this probability distribution, they cannot predict the exact outcome of the randomized selection at any specific time step t .

Furthermore, the goal of a successful policy is to guarantee sublinear cumulative regret, which means the regret grows at a slower rate than the time horizon, typically scaling as $R_n = O(\sqrt{n})$. The mathematical intuition behind sublinear regret is that the average regret incurred per time step approaches zero as the operational horizon n approaches infinity:

$$\lim_{n \rightarrow \infty} \frac{R_n}{n} = 0$$

Consequently, while the adversary can exploit the learner on individual rounds, they cannot prevent the learner from discovering the overall best static action in the long run. This asymptotic convergence proves that the randomized policy effectively hedges against adversarial manipulation over an extended period.

A concrete real-world example of this framework is **Spam Filtering and Content Security**. In this domain, spammers act as an active adversary by continuously modifying email structures, text patterns, and hyperlinks to evade detection. If the spam filter relies on a deterministic rule or policy, the adversary can quickly map out the exact criteria used for blocking and engineer a perfect bypass. By modeling the filter as a randomized adversarial bandit, the system introduces enough unpredictability to prevent spammers from finding a reliable backdoor. Intuitively, this strategy secures the system against adaptive attacks and forces the long-term filtering performance to match the best static classification model in hindsight.

Question 5: Non-Stationary Bandits

[2 Points]

Are adversarial bandit algorithms suitable for modeling non-stationary environments where reward distributions change over time? Discuss the role of the regret benchmark in your answer.

Answer:

Adversarial bandit algorithms are structurally suitable for running in non-stationary environments because they operate without making any statistical or distributional assumptions about the rewards. Since a non-stationary reward sequence is simply a special case of an arbitrary or worst-case sequence, adversarial algorithms like Exp3 will execute successfully without experiencing algorithmic or mathematical failure.

However, the primary limitation of utilizing standard adversarial algorithms in non-stationary settings lies in the design of the standard regret benchmark. The classical adversarial framework minimizes static regret, which evaluates the learner's performance against the single best static

action in hindsight:

$$R_n = \max_{a \in \mathcal{A}} \sum_{t=1}^n x_{t,a} - \mathbb{E} \left[\sum_{t=1}^n x_{t,A_t} \right]$$

Furthermore, in a truly non-stationary environment, the optimal policy shifts dynamically across different time intervals. For instance, arm 1 may be the optimal choice during the first 10 steps, while arm 2 becomes the optimal choice during the subsequent 10 steps. Intuitively, comparing an online learner against a single fixed baseline over the entire horizon n yields an uninformative and overly forgiving benchmark. Consequently, even if an algorithm achieves low static regret, it might still perform poorly compared to a dynamic oracle that switches arms as the environment changes.

Moreover, to properly handle non-stationarity, the objective must be redefined from static regret to dynamic regret (also known as tracking or shifting regret). This advanced metric benchmarks the learner against the optimal sequence of changing actions rather than a single static arm. Therefore, while the underlying adversarial mechanism can ingest non-stationary inputs, standard adversarial algorithms require modifications—such as sliding windows or discounting past rewards—to minimize the appropriate dynamic benchmark.

Question 6: Contextual Bandits

[2 Points]

In linear contextual bandit algorithms, such as LinUCB or OFUL, the context c observed at each time step restricts the available action feature vectors to a specific subset. Discuss whether this context-dependent constraint on the action space violates the underlying mathematical arguments of the algorithm. How does the policy dynamically adapt to learning different contexts over time?

Answer:

The context-dependent constraint on the action space does not violate the underlying mathematical arguments of linear contextual bandit algorithms. In standard proofs for algorithms like LinUCB or OFUL, the theoretical regret bounds are established to hold for an arbitrary, potentially adversarial sequence of decision sets $\mathcal{X}_t \subset \mathbb{R}^d$. Consequently, the framework does not assume that the learner can freely choose any feature vector in the entire vector space, nor does it require the contexts to be independent and identically distributed (i.e., i.i.d.).

The algorithm manages uncertainty using the design matrix V_t , which accumulates historical geometric information along the directions of the feature vectors that were actually selected:

$$V_t = \lambda I + \sum_{\tau=1}^{t-1} x_{\tau} x_{\tau}^{\top}$$

Furthermore, the learner's policy adapts smoothly to learning different contexts over time through the following mechanisms:

- ◊ **Subspace Learning Capability:** The algorithm does not need to learn the global parameter vector θ^* simultaneously across all dimensions. If the environment consistently presents contexts that restrict actions to a specific lower-dimensional subspace, the design matrix V_t will grow in those directions. Consequently, the learner quickly minimizes local regret within that active subspace.
- ◊ **Dynamic Uncertainty Activation:** When a novel or rare context eventually arrives, it forces the available actions to project onto previously unexplored directions in $\mathbb{R}^d$. In these unvisited directions, the eigenvalues of V_t remain small, which causes the inverse matrix

V_t^{-1} to be large. Consequently, the upper confidence bound width, given by $\beta_t \sqrt{x^\top V_t^{-1} x}$, inflates substantially for those actions.

- ◊ **Sequential Knowledge Integration:** Intuitively, it is mathematically irrelevant whether the algorithm learns all context types simultaneously or sequentially over time. The matrix V_t acts as a cumulative repository of directional confidence. Every time an action is taken, information is permanently deposited into that specific geometric orientation, naturally decreasing future uncertainty whenever a similar context reappears.

Moreover, the Optimism in the Face of Uncertainty (OFUL) principle guarantees that the learner always balances exploitation within well-known contexts and immediate exploration when encountering constrained, unfamiliar decision sets. Therefore, the structural limitations imposed by the context sequence affect only the transient rate of information collection in certain directions, without invalidating the global sublinear regret guarantees of the system.

3 Reduction

Question 1: Reduction

[5 Points]

Explain how similar or different LinUCB and UCB are for this d -armed bandit instance where actions are standard basis vectors e_i . In particular, compare the confidence bounds obtained by the two algorithms.

Answer:

When the action set is restricted to the standard basis vectors $\mathcal{A} = \{e_1, e_2, \dots, e_d\}$, LinUCB mathematically collapses to a regularized version of standard UCB. To see this relationship, we analyze the structure of the design matrix V_{t-1} at time step t . The matrix is defined as:

$$V_{t-1} = \lambda I + \sum_{\tau=1}^{t-1} x_\tau x_\tau^\top$$

Since every chosen action x_τ is a standard basis vector e_i , the outer product $e_i e_i^\top$ forms a matrix with a 1 at the (i, i) -th position and 0 elsewhere. Summing these outer products over time simply counts how many times each arm has been pulled. Let $N_i(t-1)$ denote the pull count of arm i prior to round t . The design matrix V_{t-1} becomes a purely diagonal matrix:

$$V_{t-1} = \text{diag}(\lambda + N_1(t-1), \lambda + N_2(t-1), \dots, \lambda + N_d(t-1))$$

Because the matrix is diagonal, its inverse V_{t-1}^{-1} is found by simply inverting each diagonal element:

$$V_{t-1}^{-1} = \text{diag}\left(\frac{1}{\lambda + N_1(t-1)}, \frac{1}{\lambda + N_2(t-1)}, \dots, \frac{1}{\lambda + N_d(t-1)}\right)$$

When evaluating the LinUCB exploration bonus for a specific arm $x = e_i$, the quadratic form isolates the i -th diagonal entry of the inverse matrix:

$$\text{Bonus}_{\text{LinUCB}}(e_i) = \sqrt{\beta_t} \sqrt{e_i^\top V_{t-1}^{-1} e_i} = \sqrt{\frac{\beta_t}{\lambda + N_i(t-1)}}$$

In comparison, the standard UCB algorithm computes its exploration bonus as:

$$\text{Bonus}_{\text{UCB}}(i) = c \sqrt{\frac{\ln t}{N_i(t-1)}}$$

Comparing these two expressions reveals the exact similarities and differences:

- ◊ **The Core Similarity:** In both algorithms, the confidence bound shrinks at a rate inversely proportional to the square root of the number of pulls, meaning Bonus $\propto 1/\sqrt{N_i(t-1)}$. The relative distribution of the exploration bonus across different actions remains identical under both strategies.
- ◊ **The Role of λ :** LinUCB incorporates a ridge regularization parameter $\lambda > 0$. This term naturally prevents division by zero during the initial rounds when an arm has not been pulled yet ($N_i(t-1) = 0$), smoothing out the early exploration phase without requiring a mandatory initial pull for every single arm.
- ◊ **Global Scaling Factors:** The global exploration intensity is governed by $\sqrt{\beta_t}$ in LinUCB and by $c\sqrt{\ln t}$ in standard UCB. While these scaling factors follow different theoretical trajectories over time, they act uniformly across all arms at any given time step t , preserving the relative preference order among actions.

Question 2: The Confidence Ellipsoid

[5 Points]

In LinUCB, the confidence set $\mathcal{C}_t \subset \mathbb{R}^d$ is refined as more observations are collected. Using the ridge least-squares estimator, one obtains a confidence ellipsoid of the form $\mathcal{C}_t \subseteq \mathcal{E}_t = \{\theta \in \mathbb{R}^d : \|\theta - \hat{\theta}_{t-1}\|_{V_{t-1}}^2 \leq \beta_t\}$. Give a geometric interpretation of this ellipsoid and explain how its volume changes over time.

Answer:

The mathematical expression defining the confidence set $\mathcal{E}_t$ represents a d -dimensional hyper-ellipsoid centered at the empirical ridge least-squares estimate $\hat{\theta}_{t-1}$. To understand its geometric structure, we expand the squared Mahalanobis norm using the symmetry of the design matrix $V_{t-1} = V_{t-1}^\top$:

$$\|\theta - \hat{\theta}_{t-1}\|_{V_{t-1}}^2 = \theta^\top V_{t-1} \theta - 2\theta^\top V_{t-1} \hat{\theta}_{t-1} + \hat{\theta}_{t-1}^\top V_{t-1} \hat{\theta}_{t-1} \leq \beta_t$$

This expansion manifests as a quadratic form in terms of the unknown parameter vector θ . Geometrically, the orientation and lengths of the principal axes of this ellipsoid are directly governed by the eigensystem of the matrix V_{t-1} :

- ◊ **Axis Alignment:** The eigenvectors of V_{t-1} determine the orthogonal directions of the ellipsoid's axes in $\mathbb{R}^d$.
- ◊ **Axis Lengths:** The length of each axis is inversely proportional to the square root of the corresponding eigenvalue of V_{t-1} , denoted as $\sqrt{\beta_t/\lambda_i(V_{t-1})}$. Intuitively, directions in the feature space that have been frequently explored accumulate large eigenvalues, forcing the ellipsoid to contract and become highly narrow along those specific orientations. Conversely, unexplored directions retain small eigenvalues ($\approx \lambda$), meaning the ellipsoid remains wide and elongated, reflecting high statistical uncertainty.

The volume of a d -dimensional ellipsoid scales proportionally to the inverse square root of the determinant of its defining matrix:

$$\text{Volume}(\mathcal{E}_t) \propto \frac{\beta_t^{d/2}}{\sqrt{\det(V_{t-1})}}$$

Furthermore, as the time step t increases, the learner sequentially pulls actions and adds positive semi-definite rank-one matrices ($x_t x_t^\top$) to the design matrix via $V_t = V_{t-1} + x_t x_t^\top$. By the

matrix determinant lemma, this accumulation guarantees that $\det(V_t) > \det(V_{t-1})$, meaning the denominator grows monotonically over time. Consequently, despite the controlled growth of the exploration threshold β_t , the total volume of the confidence ellipsoid strictly shrinks over time. This continuous contraction visually represents the policy narrowing down the parameter uncertainty space as it converges toward the true underlying parameter vector θ^* .

4 One-Armed Bandits

Question 1: One-Armed Bandits

[15 Points]

- (a) Let n be fixed, and let $\pi = (\pi_t)_{t=1}^n$ be an arbitrary policy. Prove that there exists a retirement policy $\pi' = (\pi'_t)_{t=1}^n$ such that, for every bandit instance $\nu \in \mathcal{E}$, $R_n(\pi', \nu) \leq R_n(\pi, \nu)$.
- (b) Let $\pi = (\pi_t)_{t=1}^\infty$ be any retirement policy. Prove that there exists a bandit instance $\nu \in \mathcal{E}$ such that $\lim_{n \rightarrow \infty} \frac{R_n(\pi, \nu)}{n} > 0$.

Answer:

(a) Proof of Policy Reduction to Retirement Policies

Let $\nu = (P_1, \delta_{\mu_2}) \in \mathcal{E}$ be a one-armed bandit instance where arm 1 has an unknown mean μ_1 and arm 2 delivers a known deterministic reward μ_2 . The expected cumulative pseudo-regret of any arbitrary policy π over a finite horizon n can be expressed solely as a function of the expected number of pulls of each arm:

$$R_n(\pi, \nu) = n \max\{\mu_1, \mu_2\} - \left(\mathbb{E}_\pi[N_1(n)]\mu_1 + \mathbb{E}_\pi[N_2(n)]\mu_2 \right)$$

where $N_i(n) = \sum_{t=1}^n \mathbb{I}(A_t = i)$ and $N_1(n) + N_2(n) = n$.

Because the reward distribution of arm 2 is the Dirac delta δ_{μ_2} , pulling arm 2 yields a deterministic reward $X_t = \mu_2$ with zero variance. Consequently, observing X_t when $A_t = 2$ provides absolutely no new statistical information regarding the unknown parameter μ_1 of arm 1. The learner can perfectly simulate the feedback of arm 2 internally without interacting with the environment.

We can therefore construct an equivalent retirement policy π' that rearranges the schedule of arm pulls. Let π' observe the history of arm 1 and sequentially pull arm 1 exactly $N_1(n)$ times at the very beginning of the horizon, where the stopping condition (and thus the distribution of $N_1(n)$) is driven by the exact same conditional probabilities as the original policy π . Once this exploration phase finishes, π' permanently switches to arm 2 for the remaining $n - N_1(n)$ rounds.

Since π' preserves the exact filtration and information gathering process from arm 1, the probability distribution of the random variable $N_1(n)$ remains identical under both policies, implying $\mathbb{E}_{\pi'}[N_1(n)] = \mathbb{E}_\pi[N_1(n)]$. Substituting this equality back into the regret definition yields:

$$R_n(\pi', \nu) = R_n(\pi, \nu) \leq R_n(\pi, \nu)$$

This proves that restricting the policy space to retirement policies does not increase the optimal achievable regret under a finite horizon.

(b) Proof of Uniform Linear Regret in Infinite Horizon

Let π be an infinite-horizon retirement policy, and let $\tau = \inf\{t \geq 1 : A_t = 2\}$ be the random variable denoting the retirement time. We can partition the sample space into two mutually exclusive and exhaustive events: $\{\tau = \infty\}$ (the policy never plays arm 2) and $\{\tau < \infty\}$ (the policy retires to arm 2 at some finite time step).

We can express the asymptotic normalized regret $\lim_{n \rightarrow \infty} \frac{R_n(\pi, \nu)}{n}$ by conditioning on these two events:

1. **Case 1: Suppose the true instance satisfies $\mu_1 < \mu_2$ (Arm 2 is optimal).** On the event $\{\tau = \infty\}$, the policy pulls the suboptimal arm 1 for all n rounds, so $\lim_{n \rightarrow \infty} \frac{N_1(n)}{n} = 1$. Each pull incurs a constant loss gap of $\Delta = \mu_2 - \mu_1 > 0$. Therefore:

$$\lim_{n \rightarrow \infty} \frac{R_n(\pi, \nu)}{n} \geq (\mu_2 - \mu_1) \cdot \mathbb{P}_{\mu_1}(\tau = \infty)$$

2. **Case 2: Suppose the true instance satisfies $\mu_1 > \mu_2$ (Arm 1 is optimal).** On the event $\{\tau < \infty\}$, the policy commits to the suboptimal arm 2 from time step τ onward. Since τ is finite, $\lim_{n \rightarrow \infty} \frac{N_2(n)}{n} = 1$. Each pull after retirement incurs a constant loss gap of $\Delta = \mu_1 - \mu_2 > 0$. Therefore:

$$\lim_{n \rightarrow \infty} \frac{R_n(\pi, \nu)}{n} \geq (\mu_1 - \mu_2) \cdot \mathbb{P}_{\mu_1}(\tau < \infty)$$

To prove that a retirement policy cannot uniformly avoid linear regret over Bernoulli parameters $\mu_1 \in [0, 1]$, we utilize a measure change argument. Consider a baseline environment where $\mu_1 = \mu_2$. Since $\mathbb{P}_{\mu_2}(\tau = \infty) + \mathbb{P}_{\mu_2}(\tau < \infty) = 1$, at least one of these probabilities must be strictly positive:

- ◊ If $\mathbb{P}_{\mu_2}(\tau = \infty) > 0$, we can perturb the environment to an adversarial instance ν^- where $\mu_1 = \mu_2 - \epsilon < \mu_2$. By the absolute continuity of Bernoulli measures over finite paths, the probability of never retiring remains strictly positive under the perturbed distribution, meaning $\mathbb{P}_{\mu_1}(\tau = \infty) > 0$. This results in a strictly linear regret bound: $\lim_{n \rightarrow \infty} \frac{R_n(\pi, \nu^-)}{n} \geq \epsilon \cdot \mathbb{P}_{\mu_1}(\tau = \infty) > 0$.
- ◊ If $\mathbb{P}_{\mu_2}(\tau < \infty) > 0$, we can perturb the environment to an alternate instance ν^+ where $\mu_1 = \mu_2 + \epsilon > \mu_2$. Similarly, the probability of early retirement remains strictly bounded away from zero, implying $\mathbb{P}_{\mu_1}(\tau < \infty) > 0$. This forces the policy to get trapped on arm 2 forever, yielding a linear regret bound: $\lim_{n \rightarrow \infty} \frac{R_n(\pi, \nu^+)}{n} \geq \epsilon \cdot \mathbb{P}_{\mu_1}(\tau < \infty) > 0$.

Thus, for any retirement policy π , there always exists a Bernoulli parameter $\mu_1 \in [0, 1]$ that yields a worst-case linear regret scaling:

$$\sup_{\nu \in \mathcal{E}} \lim_{n \rightarrow \infty} \frac{R_n(\pi, \nu)}{n} > 0$$

5 Mirror Descent and FTRL

5.1 Legendre Regularizers

Question 1: Legendre Functions

[3 Points]

For each of the following real-valued functions, decide whether it is Legendre on the given domain. For each case, briefly justify your answer by discussing convexity, differentiability on the interior, and the behavior of the gradient near the boundary.

Answer:

A function f is Legendre if it is strictly convex, continuously differentiable on the interior of its domain, and its gradient's magnitude approaches infinity as x approaches any finite boundary point.

(a) $f(x) = x^2$ on $[-1, 1]$:

Not Legendre. While it is strictly convex ($f''(x) = 2 > 0$) and differentiable on $(-1, 1)$, its gradient $f'(x) = 2x$ does not explode at the boundaries. As $x \rightarrow 1$, $f'(x) \rightarrow 2$, and as $x \rightarrow -1$, $f'(x) \rightarrow -2$, which are finite values.

(b) $f(x) = -\sqrt{x}$ on $[0, \infty)$:

Legendre. The function is strictly convex since $f''(x) = \frac{1}{4x^{3/2}} > 0$ for $x > 0$. It is differentiable on the interior $(0, \infty)$. The only finite boundary point is $x = 0$. As $x \rightarrow 0^+$, the gradient $f'(x) = -\frac{1}{2\sqrt{x}}$ approaches $-\infty$, satisfying the boundary condition.

(c) $f(x) = \log(1/x)$ on $[0, \infty)$:

Legendre. Rewriting as $f(x) = -\log x$, it is strictly convex ($f''(x) = 1/x^2 > 0$) and differentiable on $(0, \infty)$. At the finite boundary $x = 0$, the gradient $f'(x) = -1/x$ explodes to $-\infty$ as $x \rightarrow 0^+$.

(d) $f(x) = x \log x$ on $[0, \infty)$:

Legendre. It is strictly convex because $f''(x) = 1/x > 0$ for $x > 0$, and differentiable on the interior. The gradient is $f'(x) = \log x + 1$. As $x \rightarrow 0^+$ (the finite boundary), $\log x \rightarrow -\infty$, meaning the gradient explodes.

(e) $f(x) = |x|$ on $\mathbb{R}$:

Not Legendre. The interior of the domain is the entire real line $\mathbb{R}$. The function is not differentiable at $x = 0$, which is an interior point. Thus, it fails the differentiability requirement.

(f) $f(x) = \max\{|x|, x^2\}$ on $\mathbb{R}$:

Not Legendre. The domain interior is $\mathbb{R}$. This function has non-differentiable "kinks" at $x = 0$ (due to $|x|$) and at $x = 1, -1$ (where $|x|$ intersects x^2). Since it is not continuously differentiable everywhere on the interior, it is not Legendre.

5.2 Follow-the-Leader, FTRL, and Mirror Descent

Question 2: Failure of Follow-The-Leader

[3 Points]

Let the action space be $\mathcal{A} = [-1, 1]$. Consider the loss sequence defined as $y_1 = 0.5$, and for $t \geq 2$, $y_t = 1$ if t is odd, and $y_t = -1$ if t is even. Prove that the Follow-The-Leader (FTL) algorithm suffers linear regret on this sequence.

Answer:

The Follow-The-Leader (FTL) algorithm chooses the action a_t that minimizes the cumulative loss observed so far:

$$a_t = \arg \min_{a \in [-1, 1]} \sum_{s=1}^{t-1} a \cdot y_s = \arg \min_{a \in [-1, 1]} a \cdot L_{t-1}$$

where $L_{t-1} = \sum_{s=1}^{t-1} y_s$. To understand FTL's behavior, we compute the historical sum L_{t-1} at each step $t \geq 2$, by induction, the historical sum perpetually oscillates between $L_{t-1} = 0.5$ (for even t) and $L_{t-1} = -0.5$ (for odd t). Consequently, FTL perpetually plays $a_t = -1$ on even steps and $a_t = 1$ on odd steps.

However, the actual environment plays $y_t = -1$ on even steps and $y_t = 1$ on odd steps. This means FTL is strictly out of phase with the environment, and its instantaneous loss at any step

$t \geq 2$ is always maximally penalized:

$$\text{Loss}_{FTL}(t) = a_t \cdot y_t = (-1)(-1) = 1 \quad (\text{for even } t)$$

$$\text{Loss}_{FTL}(t) = a_t \cdot y_t = (1)(1) = 1 \quad (\text{for odd } t)$$

Assuming $a_1 = 0$ (arbitrary initialization with zero loss), the cumulative loss of FTL over T rounds is $\sum_{t=1}^T a_t y_t = T - 1$.

Now, we compare this against the best fixed action in hindsight, denoted as $a^* \in [-1, 1]$. The cumulative loss of any fixed action a over T steps is:

$$\text{Loss}_{Fixed}(a) = a \sum_{t=1}^T y_t$$

Since the sequence y_t alternates, the sum $\sum_{t=1}^T y_t$ is equal to 0.5 if T is odd, and -0.5 if T is even. By choosing $a^* = -1$ (if T is odd) or $a^* = 1$ (if T is even), the optimal fixed loss is consistently -0.5 .

Finally, the regret of FTL is the difference between its total loss and the best fixed action's loss:

$$\text{Regret}_T(FTL) = \sum_{t=1}^T a_t y_t - \min_{a \in [-1, 1]} a \sum_{t=1}^T y_t = (T - 1) - (-0.5) = T - 0.5$$

Since the regret scales as $O(T)$, FTL strictly suffers from linear regret on this sequence.

Question 3: Adding Quadratic Regularization

[3 Points]

Now consider the Follow-The-Regularized-Leader (FTRL) algorithm with the regularizer $F(a) = a^2$ and learning rate $\eta > 0$. Derive the explicit update rule for a_t . How does the regularizer stabilize the algorithm compared to FTL?

Answer:

The FTRL algorithm chooses the action a_t by minimizing the sum of the scaled historical losses and the regularization term:

$$a_t = \arg \min_{a \in [-1, 1]} \left(\eta \sum_{s=1}^{t-1} a \cdot y_s + F(a) \right) = \arg \min_{a \in [-1, 1]} (\eta \cdot a \cdot L_{t-1} + a^2)$$

To find the minimum, we first take the derivative of the unconstrained objective function $J(a) = \eta a L_{t-1} + a^2$ with respect to a and set it to zero:

$$\frac{\partial J(a)}{\partial a} = \eta L_{t-1} + 2a = 0 \implies a^* = -\frac{\eta L_{t-1}}{2}$$

Since the action space is restricted to $[-1, 1]$, the actual update rule requires projecting this unconstrained minimum onto the valid interval:

$$a_t = \Pi_{[-1, 1]} \left(-\frac{\eta L_{t-1}}{2} \right)$$

Assuming the learning rate is reasonably small ($\eta \leq 4$), the value $-\frac{\eta L_{t-1}}{2}$ will naturally fall strictly within $[-1, 1]$, making the projection operation the identity.

Let us analyze the behavior of this update rule on the adversarial sequence where $y_1 = 0.5$ and subsequent elements alternate between 1 and -1 . As established previously, the historical sum L_{t-1} permanently oscillates:

- ◇ **For even t :** $L_{t-1} = 0.5$. The algorithm selects $a_t = -\frac{\eta(0.5)}{2} = -\frac{\eta}{4}$. The environment plays $y_t = -1$, yielding an instantaneous loss of $(-\frac{\eta}{4})(-1) = \frac{\eta}{4}$.
- ◇ **For odd t :** $L_{t-1} = -0.5$. The algorithm selects $a_t = -\frac{\eta(-0.5)}{2} = \frac{\eta}{4}$. The environment plays $y_t = 1$, yielding an instantaneous loss of $(\frac{\eta}{4})(1) = \frac{\eta}{4}$.

How the Regularizer Stabilizes the Algorithm: Unlike the unregularized FTL algorithm, which violently oscillated between the extremes of the action space ($a_t \in \{-1, 1\}$) and suffered a maximum loss of 1 per step, FTRL is constrained by the strictly convex penalty $F(a) = a^2$. This quadratic regularizer acts as an origin-centering pull (a conservative force) that heavily penalizes large action magnitudes. Consequently, FTRL no longer overreacts to the adversarial fluctuations of L_{t-1} ; instead, it takes small, bounded steps proportional to the learning rate ($\pm\eta/4$). This stabilization drastically reduces the instantaneous loss from 1 down to $\frac{\eta}{4}$, effectively breaking the linear regret curse of FTL.

Question 4: Mirror Descent (MD)

[3 Points]

Suppose we use Mirror Descent with the regularizer $F(a) = a^2$. Derive the update rule for a_t . How does the sequence of actions compare to the FTRL sequence from the previous question?

Answer:

Mirror Descent (MD) updates the action by taking a gradient step with respect to the instantaneous loss, regularized by the Bregman Divergence generated by $F(a) = a^2$. The Bregman Divergence is given by:

$$D_F(a, b) = F(a) - F(b) - \nabla F(b)(a - b) = a^2 - b^2 - 2b(a - b) = (a - b)^2$$

The MD update rule minimizes the linearized loss and the proximity term:

$$a_t = \arg \min_{a \in [-1, 1]} \left(\eta \langle a, \nabla \ell_{t-1}(a_{t-1}) \rangle + D_F(a, a_{t-1}) \right) = \arg \min_{a \in [-1, 1]} \left(\eta a y_{t-1} + (a - a_{t-1})^2 \right)$$

Setting the derivative of the unconstrained objective to zero yields:

$$\eta y_{t-1} + 2(a - a_{t-1}) = 0 \implies a_t = a_{t-1} - \frac{\eta}{2} y_{t-1}$$

Including the constraint, the exact update rule is a projection onto the feasible set:

$$a_t = \Pi_{[-1, 1]} \left(a_{t-1} - \frac{\eta}{2} y_{t-1} \right)$$

Assuming η is small enough such that the values stay strictly within $[-1, 1]$, let us trace the sequence of actions starting from $a_1 = 0$:

- ◇ $t = 2$: $y_1 = 0.5 \implies a_2 = 0 - \frac{\eta}{2}(0.5) = -\frac{\eta}{4}$
- ◇ $t = 3$: $y_2 = -1 \implies a_3 = -\frac{\eta}{4} - \frac{\eta}{2}(-1) = -\frac{\eta}{4} + \frac{\eta}{2} = \frac{\eta}{4}$
- ◇ $t = 4$: $y_3 = 1 \implies a_4 = \frac{\eta}{4} - \frac{\eta}{2}(1) = \frac{\eta}{4} - \frac{\eta}{2} = -\frac{\eta}{4}$

Comparison with FTRL: The sequence of actions chosen by Mirror Descent identically matches the sequence produced by FTRL in Question 3. Specifically, $a_t = -\frac{\eta}{4}$ for even t , and $a_t = \frac{\eta}{4}$ for odd t .

This mathematical equivalence is not a coincidence. When the loss functions are linear ($\ell_t(a) = y_t a$) and the regularizer is purely quadratic ($F(a) = a^2$), FTRL and MD (often called

Online Gradient Descent in this context) are completely equivalent algorithms as long as the unconstrained updates do not cross the boundary of the feasible set. In FTRL, the centering force is applied globally to the historical sum, while in MD, it is applied locally via the Bregman divergence; for quadratic functions, these two regularizations perfectly align.

5.3 FTRL for Adversarial Bandits

Question 5: The FTRL Distribution

[4 Points]

Let $L_{t-1,i} = \sum_{s=1}^{t-1} \hat{Y}_{si}$ be the cumulative estimated loss of arm i before round t . Show that the FTRL distribution satisfies

$$P_{ti} = (\lambda_t + \eta L_{t-1,i})^{-2}$$

where $\lambda_t \in \mathbb{R}$ is chosen so that $P_t \in \mathcal{P}_{k-1}$. Briefly explain why such a value of λ_t exists and is unique.

Answer:

The FTRL objective at round t requires us to find $p \in \mathcal{P}_{k-1}$ that minimizes:

$$J(p) = \eta \sum_{s=1}^{t-1} \langle p, \hat{Y}_s \rangle + F(p) = \eta \langle p, L_{t-1} \rangle - 2 \sum_{i=1}^k \sqrt{p_i}$$

We can expand the inner product to express the objective purely in terms of the individual coordinates p_i :

$$J(p) = \sum_{i=1}^k (\eta p_i L_{t-1,i} - 2\sqrt{p_i})$$

Since p must be a valid probability distribution, we must enforce the equality constraint $\sum_{i=1}^k p_i = 1$ and the inequality constraints $p_i \geq 0$. We construct the Lagrangian incorporating the equality constraint using the multiplier λ_t :

$$\mathcal{L}(p, \lambda_t) = \sum_{i=1}^k (\eta p_i L_{t-1,i} - 2\sqrt{p_i}) + \lambda_t \left(\sum_{i=1}^k p_i - 1 \right)$$

To find the optimal P_t , we take the partial derivative of the Lagrangian with respect to p_i and set it to zero:

$$\frac{\partial \mathcal{L}}{\partial p_i} = \eta L_{t-1,i} - p_i^{-1/2} + \lambda_t = 0$$

Rearranging the terms to solve for P_{ti} yields:

$$P_{ti}^{-1/2} = \lambda_t + \eta L_{t-1,i} \implies P_{ti} = (\lambda_t + \eta L_{t-1,i})^{-2}$$

Notice that because P_{ti} is defined as an inverse square, the non-negativity constraint ($p_i \geq 0$) is naturally and strictly satisfied ($P_{ti} > 0$), meaning the associated KKT multipliers for the inequality constraints are zero, justifying our simplification.

Existence and Uniqueness of λ_t : To ensure $P_t \in \mathcal{P}_{k-1}$, the probabilities must sum to 1. Thus, λ_t must be the root of the following function:

$$g(\lambda) = \sum_{i=1}^k (\lambda + \eta L_{t-1,i})^{-2} = 1$$

Let us analyze the behavior of $g(\lambda)$ on the domain $\lambda \in (-\min_i(\eta L_{t-1,i}), \infty)$. In this domain, all terms $(\lambda + \eta L_{t-1,i})$ are strictly positive. As λ increases, each term $(\lambda + \eta L_{t-1,i})^{-2}$ strictly decreases. Therefore, $g(\lambda)$ is a strictly monotonically decreasing and continuous function. Furthermore, we have the limits:

$$\lim_{\lambda \rightarrow (-\min_i(\eta L_{t-1,i}))^+} g(\lambda) = \infty \quad \text{and} \quad \lim_{\lambda \rightarrow \infty} g(\lambda) = 0$$

Since the continuous function strictly decreases from ∞ to 0, by the Intermediate Value Theorem, it must cross the value 1 exactly once. Therefore, there exists a unique λ_t that satisfies the normalization condition.

Question 6: Effect of Observing a Loss

[2 Points]

Show that the probability assigned to the selected arm cannot increase immediately after observing its loss. More precisely, prove that

$$P_{t+1,A_t} \leq P_{t,A_t}$$

for $t = 1, \dots, n-1$. Explain the intuition behind this property.

Answer:

Let $j = A_t$ be the arm selected at round t . According to the importance-weighted estimator, only the estimated loss for arm j is updated, while the others remain unchanged:

$$L_{t,j} = L_{t-1,j} + \hat{Y}_{tj} \quad \text{and} \quad L_{t,i} = L_{t-1,i} \quad \text{for } i \neq j$$

Since the losses are non-negative ($y_{tj} \geq 0$), we have $\hat{Y}_{tj} \geq 0$, which implies $L_{t,j} \geq L_{t-1,j}$. We proceed by contradiction. Suppose the probability of the selected arm strictly increases after observing the loss: $P_{t+1,j} > P_{t,j}$. Using the formula derived in Question 5, this means:

$$(\lambda_{t+1} + \eta L_{t,j})^{-2} > (\lambda_t + \eta L_{t-1,j})^{-2}$$

Since all terms inside the parentheses are strictly positive, this inequality simplifies to:

$$\lambda_{t+1} + \eta L_{t,j} < \lambda_t + \eta L_{t-1,j}$$

Rearranging the terms gives:

$$\lambda_{t+1} - \lambda_t < \eta(L_{t-1,j} - L_{t,j}) \leq 0 \implies \lambda_{t+1} < \lambda_t$$

Now, let us analyze the probabilities of all the other arms $i \neq j$. Since their cumulative losses did not change ($L_{t,i} = L_{t-1,i}$) and $\lambda_{t+1} < \lambda_t$, the denominator for each unselected arm strictly decreases, causing their probabilities to strictly increase:

$$P_{t+1,i} = (\lambda_{t+1} + \eta L_{t,i})^{-2} > (\lambda_t + \eta L_{t-1,i})^{-2} = P_{t,i} \quad \text{for all } i \neq j$$

Now, if we sum the probabilities of all arms at round $t+1$, we get:

$$\sum_{k=1}^k P_{t+1,k} = P_{t+1,j} + \sum_{i \neq j} P_{t+1,i} > P_{t,j} + \sum_{i \neq j} P_{t,i} = \sum_{k=1}^k P_{t,k} = 1$$

This implies $\sum_{k=1}^k P_{t+1,k} > 1$, which strictly violates the definition of a probability distribution ($P_{t+1} \in \mathcal{P}_{k-1}$). This contradiction proves that our initial assumption must be false. Therefore, we conclude that:

$$P_{t+1,A_t} \leq P_{t,A_t}$$

Intuition: In an adversarial setting, observing a loss represents receiving a non-negative penalty. Intuitively, experiencing a penalty for taking an action should never increase the algorithm's preference (probability) for taking that exact same action in the immediately following round. The algorithm updates its internal state to be either more cautious (if the loss is positive) or indifferent (if the loss is zero), which is exactly what this property guarantees.

Question 7: Hessian of the Potential

[2 Points]

Compute the Hessian of

$$F(p) = -2 \sum_{i=1}^k \sqrt{p_i}$$

Show that

$$\nabla^2 F(p) = \frac{1}{2} \text{diag}(p^{-3/2})$$

where the power is applied coordinate-wise. Briefly explain what this Hessian says about the curvature of the regularizer near the boundary of the simplex.

Answer:

To find the Hessian matrix of the potential function $F(p)$, we first compute the first-order partial derivatives (the gradient). The function is separable, meaning each term in the sum depends only on a single coordinate p_i :

$$F(p) = -2 \sum_{i=1}^k p_i^{1/2}$$

Taking the partial derivative with respect to p_i yields:

$$\frac{\partial F}{\partial p_i} = -2 \left(\frac{1}{2} p_i^{-1/2} \right) = -p_i^{-1/2}$$

Next, we compute the second-order partial derivatives. Differentiating again with respect to p_i gives the diagonal elements of the Hessian:

$$\frac{\partial^2 F}{\partial p_i^2} = - \left(-\frac{1}{2} p_i^{-3/2} \right) = \frac{1}{2} p_i^{-3/2}$$

For the off-diagonal elements, we take the partial derivative with respect to p_j (where $i \neq j$):

$$\frac{\partial^2 F}{\partial p_i \partial p_j} = \frac{\partial}{\partial p_j} \left(-p_i^{-1/2} \right) = 0$$

Since all cross-derivatives are identically zero, the Hessian matrix is strictly diagonal. Writing this in matrix notation, we get:

$$\nabla^2 F(p) = \text{diag} \left(\frac{1}{2} p_1^{-3/2}, \frac{1}{2} p_2^{-3/2}, \dots, \frac{1}{2} p_k^{-3/2} \right) = \frac{1}{2} \text{diag}(p^{-3/2})$$

Curvature near the boundary: The Hessian matrix represents the local curvature of the function. The boundary of the probability simplex $\mathcal{P}_{k-1}$ occurs when one or more probabilities approach zero ($p_i \rightarrow 0^+$). As p_i approaches 0, the corresponding diagonal entry in the Hessian $\frac{1}{2} p_i^{-3/2}$ approaches $+\infty$.

This indicates that the regularizer has infinite curvature (it becomes infinitely steep) at the boundary. Geometrically, it acts as an extreme repulsive barrier that heavily penalizes the

algorithm for assigning a probability too close to zero to any arm. This property forces the policy to constantly explore all arms (ensuring $P_{t_i} > 0$) and prevents the premature elimination of actions, which is crucial in an adversarial bandit setting where any unplayed arm might suddenly become the optimal choice.

Question 8: Diameter of the Regularizer

[2 Points]

Show that the diameter of the simplex with respect to F satisfies

$$\text{diam}_F(\mathcal{P}_{k-1}) \leq 2\sqrt{k}$$

Here, $\text{diam}_F(\mathcal{P}_{k-1}) = \sup_{p,q \in \mathcal{P}_{k-1}} |F(p) - F(q)|$.

Answer:

To find the diameter of the simplex under $F(p) = -2 \sum_{i=1}^k \sqrt{p_i}$, we need to find the absolute difference between the global maximum and the global minimum of $F(p)$ over the probability simplex $\mathcal{P}_{k-1}$.

1. Finding the Minimum of $F(p)$: Minimizing $F(p)$ is equivalent to maximizing $\sum_{i=1}^k \sqrt{p_i}$. We can apply the Cauchy-Schwarz inequality to the vectors $u = (\sqrt{p_1}, \dots, \sqrt{p_k})$ and $v = (1, \dots, 1)$:

$$\sum_{i=1}^k \sqrt{p_i} \cdot 1 \leq \sqrt{\sum_{i=1}^k (\sqrt{p_i})^2} \sqrt{\sum_{i=1}^k 1^2} = \sqrt{\sum_{i=1}^k p_i} \sqrt{k}$$

Since $p \in \mathcal{P}_{k-1}$, we know the probabilities sum to one ($\sum_{i=1}^k p_i = 1$). Thus, $\sum_{i=1}^k \sqrt{p_i} \leq \sqrt{k}$. Multiplying by -2 reverses the inequality:

$$F(p) = -2 \sum_{i=1}^k \sqrt{p_i} \geq -2\sqrt{k}$$

This global minimum is achieved at the center of the simplex where $p_i = \frac{1}{k}$ for all i . So, $\min F(p) = -2\sqrt{k}$.

2. Finding the Maximum of $F(p)$: Maximizing $F(p)$ is equivalent to minimizing $\sum_{i=1}^k \sqrt{p_i}$. Since $p \in \mathcal{P}_{k-1}$, every coordinate satisfies $0 \leq p_i \leq 1$. For any real number in this interval, its square root is strictly greater than or equal to the number itself ($\sqrt{p_i} \geq p_i$). Summing this inequality over all coordinates yields:

$$\sum_{i=1}^k \sqrt{p_i} \geq \sum_{i=1}^k p_i = 1$$

Multiplying by -2 reverses the inequality:

$$F(p) = -2 \sum_{i=1}^k \sqrt{p_i} \leq -2(1) = -2$$

This global maximum is achieved at the extreme corners of the simplex (e.g., $p_1 = 1$, and $p_i = 0$ for $i \neq 1$). So, $\max F(p) = -2$.

3. Bounding the Diameter: The diameter is simply the difference between the maximum and minimum values of the function over the domain:

$$\text{diam}_F(\mathcal{P}_{k-1}) = \max_{p \in \mathcal{P}_{k-1}} F(p) - \min_{q \in \mathcal{P}_{k-1}} F(q) = -2 - (-2\sqrt{k}) = 2\sqrt{k} - 2$$

Since $k \geq 1$ (and trivially $k \geq 2$ in a multi-armed bandit setting), it holds mathematically that $2\sqrt{k} - 2 < 2\sqrt{k}$. Therefore, we naturally obtain the desired upper bound:

$$\text{diam}_F(\mathcal{P}_{k-1}) \leq 2\sqrt{k}$$

Question 9: Regret Bound

[4 Points]

Using the previous parts, prove that for a suitable choice of learning rate η , the regret of this FTRL algorithm satisfies

$$R_n \leq \sqrt{8kn}$$

Clearly state the value of η that you use. Briefly explain why the dependence on k and n is reasonable for adversarial bandits.

Answer:

The standard regret bound for FTRL with a twice-differentiable regularizer is given by:

$$R_n \leq \frac{\text{diam}_F(\mathcal{P}_{k-1})}{\eta} + \frac{\eta}{2} \sum_{t=1}^n \mathbb{E} \left[\hat{Y}_t^\top (\nabla^2 F(P_t))^{-1} \hat{Y}_t \right]$$

From Question 8, we established the diameter bound: $\text{diam}_F(\mathcal{P}_{k-1}) \leq 2\sqrt{k}$.

Next, we bound the local norm term. From Question 7, the Hessian is $\nabla^2 F(P_t) = \frac{1}{2} \text{diag}(P_t^{-3/2})$. Its inverse is simply:

$$(\nabla^2 F(P_t))^{-1} = 2 \text{diag}(P_t^{3/2})$$

We expand the quadratic form using the importance-weighted estimator $\hat{Y}_{ti} = \frac{\mathbb{I}\{A_t=i\}y_{ti}}{P_{ti}}$:

$$\hat{Y}_t^\top (\nabla^2 F(P_t))^{-1} \hat{Y}_t = \sum_{i=1}^k 2P_{ti}^{3/2} \hat{Y}_{ti}^2 = \sum_{i=1}^k 2P_{ti}^{3/2} \left(\frac{\mathbb{I}\{A_t=i\}y_{ti}^2}{P_{ti}^2} \right)$$

Taking the expected value conditional on P_t , and using the fact that $\mathbb{E}[\mathbb{I}\{A_t=i\}] = P_{ti}$:

$$\mathbb{E}_t \left[\sum_{i=1}^k 2P_{ti}^{3/2} \frac{\mathbb{I}\{A_t=i\}y_{ti}^2}{P_{ti}^2} \right] = \sum_{i=1}^k 2P_{ti}^{3/2} \frac{P_{ti}y_{ti}^2}{P_{ti}^2} = 2 \sum_{i=1}^k P_{ti}^{1/2} y_{ti}^2$$

Since the true losses are bounded ($y_{ti} \in [0, 1]$), we have $y_{ti}^2 \leq 1$. Furthermore, using the Cauchy-Schwarz inequality (as shown in Question 8), we know $\sum_{i=1}^k \sqrt{P_{ti}} \leq \sqrt{k}$. Thus:

$$2 \sum_{i=1}^k \sqrt{P_{ti}} y_{ti}^2 \leq 2 \sum_{i=1}^k \sqrt{P_{ti}} \leq 2\sqrt{k}$$

Substituting this back into the total regret bound yields:

$$R_n \leq \frac{2\sqrt{k}}{\eta} + \frac{\eta}{2} \sum_{t=1}^n (2\sqrt{k}) = \frac{2\sqrt{k}}{\eta} + \eta n \sqrt{k}$$

To minimize the bound, we optimize the learning rate η by taking the derivative with respect to η and setting it to zero (or simply equating the two terms):

$$-\frac{2\sqrt{k}}{\eta^2} + n\sqrt{k} = 0 \implies \eta^2 = \frac{2}{n} \implies \eta = \sqrt{\frac{2}{n}}$$

Finally, plugging $\eta = \sqrt{2/n}$ into the regret expression gives:

$$R_n \leq \frac{2\sqrt{k}}{\sqrt{2/n}} + \sqrt{\frac{2}{n}} n \sqrt{k} = \sqrt{2kn} + \sqrt{2kn} = 2\sqrt{2kn} = \sqrt{8kn}$$

Why this dependence is reasonable: The $O(\sqrt{n})$ dependence is the standard minimax optimal rate for online learning, ensuring that the average regret approaches zero as time goes to infinity. The $\sqrt{k}$ dependence is the fundamental cost of partial feedback in adversarial bandits. Because we only observe 1 out of k arms, the variance of our importance-weighted estimator inherently scales with k . This leads to the $\sqrt{k}$ penalty, contrasting with the $O(\log k)$ penalty achievable in full-information settings (like Hedge/Multiplicative Weights).

Question 10: Mirror Descent Instead of FTRL

[2 Points]

Suppose we use Mirror Descent with the same potential $F(p) = -2 \sum_{i=1}^k \sqrt{p_i}$ instead of FTRL. Are the resulting algorithms the same? If not, explain the difference between the two update rules. What regret guarantee can be proved for Mirror Descent in this setting?

Answer:

No, the resulting algorithms are not the same. While they were equivalent in Question 4 under an unconstrained quadratic regularizer, the extreme non-linearity of $F(p) = -2 \sum_{i=1}^k \sqrt{p_i}$ breaks this equivalence.

Difference in Update Rules:

- ◇ **FTRL (Global Memory):** The FTRL algorithm maintains the exact, unprojected sum of all historical estimated losses ($L_{t-1} = \sum_{s=1}^{t-1} \hat{Y}_s$). It solves a single global optimization problem at each step, mapping this pristine history directly onto the probability simplex using the regularizer $F(p)$.
- ◇ **Mirror Descent (Local Steps):** MD, on the other hand, performs a local update step. It tries to minimize the instantaneous loss $\hat{Y}_{t-1}$ while staying close to the *previous* probability distribution P_{t-1} using the Bregman divergence $D_F(p, P_{t-1})$. Because it must project the intermediate result back onto the simplex at every single step, the historical gradients are filtered through repeated projections.

Regret Guarantee for Mirror Descent: As proven in Question 7, the Hessian of this specific regularizer approaches infinity near the boundaries of the simplex. Consequently, the Bregman divergence severely distorts the gradient updates during MD's step-by-step projection process. This continuous distortion causes MD to "forget" crucial information from earlier rounds. As a result, Mirror Descent with this potential fails to achieve the optimal $O(\sqrt{kn})$ regret bound that we proved for FTRL. In adversarial bandits, this form of Mirror Descent is unstable and generally suffers from suboptimal, often linear, regret.

Question 11: Implementation of FTRL for Adversarial Bandits

[2 Points]

Explain how you would implement the FTRL algorithm described above. Your answer should include:

- ◇ how to compute the distribution P_t at each round;
- ◇ how to sample the action A_t ;

- ◇ how to construct the importance-weighted estimator $\hat{Y}_t$;
- ◇ how to update the cumulative estimated losses.

In particular, describe how to solve the one-dimensional normalization equation

$$\sum_{i=1}^k (\lambda_t + \eta L_{t-1,i})^{-2} = 1$$

Answer:

To practically implement the adversarial FTRL algorithm in a programming environment (e.g., Python), the steps at each round t would be executed as follows:

1. Computing the Distribution P_t and Solving for λ_t : Since $P_{ti} = (\lambda_t + \eta L_{t-1,i})^{-2}$, we must first find λ_t such that the probabilities sum to 1. Because the function $g(\lambda) = \sum_{i=1}^k (\lambda + \eta L_{t-1,i})^{-2} - 1$ is strictly monotonically decreasing and continuously differentiable on its domain, it cannot be solved analytically. Instead, we use a 1D numerical root-finding algorithm such as the **Bisection Method** or **Newton-Raphson method**.

For the Bisection method, we define the search interval $[\lambda_{min}, \lambda_{max}]$. The lower bound must avoid the asymptote: $\lambda_{min} = -\min_i(\eta L_{t-1,i}) + \epsilon$ (where $\epsilon > 0$ is a small threshold). The upper bound λ_{max} can be found by exponentially scaling a value until $g(\lambda) < 0$. Once the root-finding algorithm converges to λ_t , we compute the final array P_t using the analytical formula.

2. Sampling the Action A_t : With the valid probability distribution vector P_t computed, we sample an action index $A_t \in \{1, 2, \dots, k\}$. In code, this is implemented using a pseudo-random number generator that samples from a categorical distribution (e.g., `numpy.random.choice(k, p=P_t)`).

3. Constructing the Estimator $\hat{Y}_t$: After pulling arm A_t , the environment reveals the true scalar loss y_{t,A_t} . We construct the estimated loss vector $\hat{Y}_t$ of size k by initializing a vector of zeros. We then update only the index corresponding to the pulled arm:

$$\hat{Y}_{t,A_t} = \frac{y_{t,A_t}}{P_{t,A_t}}$$

All other entries $i \neq A_t$ remain exactly 0.

4. Updating the Cumulative Losses: Finally, we maintain the state for the next round by updating the cumulative loss vector. This is a simple vector addition:

$$L_t = L_{t-1} + \hat{Y}_t$$

Since only one entry in $\hat{Y}_t$ is non-zero, this operation simply adds the estimated loss of the selected arm to its respective cumulative tally, leaving the history of all unselected arms unchanged.

6 Contextual Bandits for Text Moderation

Implementation 1: Notebook Implementation

[20 Points]

Complete all notebook cells marked with TODO. Your grade will be based on correctness, completeness, and the quality of your experimental analysis.

Answer:

See Ctx_Bandits.ipynb.

Question 1: Analysis Questions

[5 Points]

Answer the following questions based on your implementations and experimental results.

(1) Dataset

- (a) What are the limitations of using binary labels as reward ground truth?
- (b) Why is `tweet_eval/hate` a reasonable but imperfect OOD stress test for `tweet_eval/offensive`?

(2) Severity Proxy

- (a) Why is a severity proxy dangerous if treated as ground truth?
- (b) What signals does it overemphasize?

(3) Reward, Oracle, Regret, and Cost Sensitivity

- (a) Which product profile is most likely to over-block?
- (b) Why is this a bandit problem rather than a standard classification problem?
- (c) Why does the oracle action distribution change across profiles?

(4) Logged Bandit Data

- (a) How might low propensities affect offline policy evaluation?

(5) Risk Model for Safety and Calibration

- (a) Is it fair to use labels here while the reward model uses bandit feedback?
- (b) What would change if labels were delayed or missing?

(6) Fixed Policies from the Reward Model

- (a) Does uncertainty help?
- (b) Which policy has the highest block rate?
- (c) Which policy has the worst OOD degradation?

(7) Safe Exploration Constraints

- (a) What is an ethically acceptable exploration space for content moderation?

(8) Online LinUCB

- (a) Why can LinUCB be more stable than neural Thompson sampling during the early stages of learning?

(9) Offline Policy Evaluation

- (a) Which estimator is most variable?
- (b) Which estimator is most biased when the reward model is poor?
- (c) What happens when target actions have very low behavior propensities?

(10) Calibration

- (a) How does miscalibration affect safe exploration?

Answer:

(1) Dataset

- (a) Binary labels fail to capture the spectrum of toxicity. In moderation, the operational cost of allowing a mild insult is significantly different from allowing a severe physical threat. Binary labels prevent the formulation of an accurate, cost-sensitive reward matrix that distinguishes between intermediate penalizations.
- (b) It is reasonable because hate speech shares aggressive vocabulary, toxic semantic structures, and abusive patterns with general offensive language. It is imperfect because there is a distinct distribution shift; the `harmful_rate` jumps from 31.7% in the in-domain data to 42.4% in the OOD data. Additionally, hate speech specifically targets identity groups, which introduces out-of-vocabulary features that the offensive-trained model has not seen, leading to degraded calibration (ECE increases from 0.259 to 0.355).

(2) Severity Proxy

- (a) It relies on a heuristic combination of predefined terms and text length. Treating it as absolute ground truth is dangerous because it can penalize benign but edgy text, while simultaneously failing to capture implicit, contextual, or sarcastic hate speech that avoids specific forbidden keywords.
- (b) According to the implemented proxy function, it heavily overemphasizes explicit profanity (e.g., words in `MILD_RISK_TERMS`), specific special characters like hashtags (`#`) and mentions (`@`), and penalizes shorter texts disproportionately due to the square root length normalization.

(3) Reward, Oracle, Regret, and Cost Sensitivity

- (a) The `high_safety` profile is mathematically designed to over-block. It imposes severe penalties for allowing harmful content (e.g., a base penalty of -3.20 compared to -1.20 in `free_expression`), forcing the decision boundary to heavily favor blocking even when uncertainty is present.
- (b) In classification, the algorithm receives full feedback (the true label of the post). In this setting, the algorithm only observes the reward for the specific action it chose (partial feedback), mimicking the real-world scenario where the true outcome of an unchosen moderation action remains unknown. Furthermore, different errors (false positives vs. false negatives) have asymmetric, action-dependent costs.

- (c) The oracle action distribution shifts because the utility of each action is a function of the specific cost profile’s coefficients. As the penalty for missing harmful content increases (e.g., in **high_safety**), the expected reward for **allow** drops below **block** for edge cases, causing the oracle to dynamically shift its optimal actions toward safer interventions.

(4) Logged Bandit Data

- (a) Low propensities (e.g., observing a minimum propensity of 0.033) cause the importance weights ($\frac{1}{p_i}$) to explode. In Offline Policy Evaluation (OPE), this leads to unbounded variance, making estimators like Inverse Propensity Scoring (IPS) highly unstable and unreliable.

(5) Risk Model for Safety and Calibration

- (a) Yes, it is fair. In a real deployment, a risk model can be trained on a separate, highly-curated historical dataset audited by human moderators (full supervision), while the reward model continuously learns from ongoing, cost-sensitive bandit interactions (partial feedback).
- (b) If labels are delayed or missing, the risk classifier becomes stale or biased. Consequently, the model’s predicted probabilities will misalign with empirical reality (poor calibration). The safe exploration thresholds would then either trigger too often (starving the model of exploration) or fail to trigger (leaking toxic content to users).

(6) Fixed Policies from the Reward Model

- (a) In a purely offline, fixed evaluation environment without online updates, uncertainty does not help and can actively hurt average regret. As shown in the evaluation tables, the deterministic **greedy_mlp** achieved an average regret of 0.950, whereas policies injecting uncertainty like **dropout_ts_mlp** (0.970) and **ensemble_ts_mlp** (0.954) performed worse.
- (b) Based on the in-domain evaluation results, the **epsilon_0.15_mlp** policy exhibits the highest block rate at 0.1585.
- (c) According to the unconstrained policy degradation table, **ensemble_ts_mlp** suffered the worst OOD degradation, with a regret increase of 0.1874.

(7) Safe Exploration Constraints

- (a) Ethically, a platform cannot randomly publish potentially toxic content to users simply to learn its reward. The acceptable exploration space strictly masks out the **allow** action when the predicted risk exceeds an **unsafe_threshold**, ensuring exploration is limited to administrative actions (**review** or **block**) for high-risk content.

(8) Online LinUCB

- (a) LinUCB uses Ridge Regression with a closed-form, deterministic matrix inversion (Sherman-Morrison formula) to compute exact upper confidence bounds based on features. Neural TS, conversely, relies on stochastic gradient updates, which suffer from high variance, initialization sensitivity, and slow representation learning during the critical early steps.

(9) Offline Policy Evaluation

- (a) The Inverse Propensity Scoring (IPS) estimator is the most variable because it relies purely on importance weights ($\frac{\pi(a|x)}{p_i}$) which explode when the logging policy propensity p_i is close to zero.

- (b) The Direct Method (DM) is the most biased if the reward model is poor. It completely ignores the actual logged rewards and calculates value based entirely on the reward model's estimations ($\hat{Q}$), directly inheriting any bias present in the network.
- (c) When target actions have very low behavior propensities, the importance weights in both IPS and Doubly Robust (DR) estimators become massive. This causes the estimated policy value to swing wildly, rendering the evaluation highly unreliable.

(10) Calibration

- (a) Safe exploration relies on hard thresholds (e.g., masking actions if risk > 0.40). The threshold analysis shows that at a predicted risk of 0.20, the empirical harm rate is actually over 0.45. This severe underestimation (miscalibration) means the safety mask will fail to trigger when it should, exposing the platform to toxic content. Conversely, overestimating risk would lock the model into a safe state, preventing necessary exploration.